

Chapter 5

C Functions

An essential structuring unit in C

Chapter Targets

- Construct programs modularly from small pieces called functions
- Use common math functions in the C standard library
- Create new functions
- Use the mechanism that pass information between functions
- Learn how the call/return mechanism is supported by The Function Call Stack and stack frames
- Use simulation techniques based on random number generator
- Write and use functions that call themselves

Introduction

- Most computer programs that solve real-world problems are much larger than the programs presented in the first few chapters.
- Experience has shown that the best way to develop and maintain a large program is to construct it from smaller pieces, each of which is more manageable than the original program.
- This technique is called **divide and conquer**.
- This chapter describes some key features of the C language that facilitate the design, implementation, operation and maintenance of large programs.

Modularizing Programs in C

- Functions are used to modularize programs
- C programs are typically written by combining new functions you write with prepackaged functions available in the C standard library.
- The C standard library provides a rich collection of functions for performing common mathematical calculations, string manipulations, character manipulations, input/output, and many other useful operations.
- The C language and the standard library are both specified by the C standard.

Modularizing Programs in C (cont'd)



Good Programming Practice 5.1

Familiarize yourself with the rich collection of functions in the C standard library.



Software Engineering Observation 5.1

Avoid reinventing the wheel. When possible, use C standard library functions instead of writing new functions. This can reduce program development time. These functions are written by experts, well-tested and efficient.



Portability Tip 5.1

Using the functions in the C standard library helps make programs more portable.

Modularizing Programs in C (cont'd)

- The functions `printf`, `scanf` and `pow` that we've used in previous chapters are standard library functions.
- You can write your own functions to define tasks that may be used at many points in a program.
- These are sometimes referred to as **programmer-defined functions**.
- The statements defining the function are written only once, and the statements are hidden from other functions.
- Functions are invoked by a **function call**, which specifies the **function name** and provides information (as **arguments**) that the called function needs to perform its designated task.

Modularizing Programs in C (cont'd)

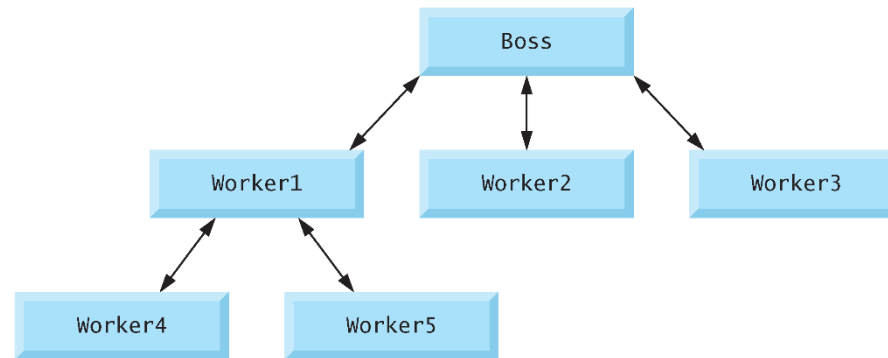
- A common analogy for this is the **hierarchical form** of management.
- A boss (the **calling function** or **caller**) asks a worker (the **called function**) to perform a task and report back when the task is done.
- For example, a function needing to display information on the screen **calls** the worker function `printf` to perform that task, then `printf` displays the information and reports back—or **returns**—to the calling function when its task is completed.
- The boss function **does not know how** the worker function performs its designated tasks.

Modularizing Programs in C (cont'd)

- The worker may call other worker functions, and the boss will be unaware of this.
- We'll soon see how this “**hiding**” of **implementation details** promotes good software engineering.

Modularizing Programs in C.

- Hierarchical boss-function/worker-function relationship



- A boss function communicates with several worker functions in a hierarchical manner.
- Note that worker1 acts as a boss function to worker4 and worker5.

Math Library Functions

- Math library functions allow you to perform certain **common mathematical calculations**.
- Functions are normally used in a program by writing the **name of the function** followed by a **left parenthesis** followed by the **argument** (or a comma-separated list of arguments) of the function followed by a **right parenthesis**.
- For example, a programmer desiring to calculate and print the square root of 900.0 you might write

```
printf("%.2f", sqrt(900.0));
```


When this statement executes, the math library function `sqrt` is called to calculate the square root of the number contained in the parentheses (`900.0`).

Math Library Functions (cont'd)

- The number `900.0` is the argument of the `sqrt` function.
- The preceding statement would print `30.00`.
- The `sqrt` function takes an argument of type `double` and returns a result of type `double`.
- All functions in the math library that return floating-point values return the data type `double`.
- Note that double values, like float values, can be output using the `%f` conversion specification.



Error-Prevention Tip 5.1

Include the math header by using the preprocessor directive `#include <math.h>` when using functions in the math library.

Math Library Functions (cont'd)

- Function arguments may be **constants**, **variables**, or **expressions**.
- If $c1 = 13.0$, $d = 3.0$ and $f = 4.0$, then the statement

```
printf("%.2f", sqrt(c1 + d * f));
```

- calculates and prints the square root of $13.0 + 3.0 * 4.0 = 25.0$, namely 5.00.

Math Library Functions (cont'd)

Function	Description	Example
<code>sqrt(x)</code>	square root of x	<code>sqrt(900.0)</code> is 30.0 <code>sqrt(9.0)</code> is 3.0
<code>cbrt(x)</code>	cube root of x (C99 and C11 only)	<code>cbrt(27.0)</code> is 3.0 <code>cbrt(-8.0)</code> is -2.0
<code>exp(x)</code>	exponential function e^x	<code>exp(1.0)</code> is 2.718282 <code>exp(2.0)</code> is 7.389056
<code>log(x)</code>	natural logarithm of x (base e)	<code>log(2.718282)</code> is 1.0 <code>log(7.389056)</code> is 2.0
<code>log10(x)</code>	logarithm of x (base 10)	<code>log10(1.0)</code> is 0.0 <code>log10(10.0)</code> is 1.0 <code>log10(100.0)</code> is 2.0
<code>fabs(x)</code>	absolute value of x as a floating-point number	<code>fabs(13.5)</code> is 13.5 <code>fabs(0.0)</code> is 0.0 <code>fabs(-13.5)</code> is 13.5
<code>ceil(x)</code>	rounds x to the smallest integer not less than x	<code>ceil(9.2)</code> is 10.0 <code>ceil(-9.8)</code> is -9.0

Math Library Functions (cont'd)

Function	Description	Example
<code>floor(x)</code>	rounds x to the largest integer not greater than x	<code>floor(9.2)</code> is 9.0 <code>floor(-9.8)</code> is -10.0
<code>pow(x, y)</code>	x raised to power y (x^y)	<code>pow(2, 7)</code> is 128.0 <code>pow(9, .5)</code> is 3.0
<code>fmod(x, y)</code>	remainder of x/y as a floating-point number	<code>fmod(13.657, 2.333)</code> is 1.992
<code>sin(x)</code>	trigonometric sine of x (x in radians)	<code>sin(0.0)</code> is 0.0
<code>cos(x)</code>	trigonometric cosine of x (x in radians)	<code>cos(0.0)</code> is 1.0
<code>tan(x)</code>	trigonometric tangent of x (x in radians)	<code>tan(0.0)</code> is 0.0

Functions

- Functions allow you to modularize a program.
- All variables defined in function definitions are local variables—they can be accessed only in the function in which they're defined.
- Most functions have a list of parameters that provide the means for communicating information between functions.
- A function's parameters are also local variables of that function.



Software Engineering Observation 5.2

In programs containing many functions, main is often implemented as a group of calls to functions that perform the bulk of the program's work.

Functions (cont'd)

- Motivations for “functionalizing” a program
 - The **divide-and-conquer** approach makes program development more manageable.
 - Another motivation is software **reusability**—using existing functions as building blocks to create new programs.
 - Software reusability is a **major factor in the object-oriented programming** movement that you’ll learn more about when you study languages derived from C, such as C++, Java and C# (pronounced “C sharp”).
 - With good function naming and definition, programs can be created from standardized functions, rather than customized code. This is known as **abstraction**.
 - We use abstraction each time we use standard library functions like `printf`, `scanf` and `pow`.

Functions.

- A third motivation is to **avoid repeating code** in a program.
- Packaging code as a function allows the code to be executed from other locations in a program simply **by calling the function**.



Software Engineering Observation 5.3

Each function should be limited to performing a single, well-defined task, and the function name should express that task. This facilitates abstraction and promotes software reusability.



Software Engineering Observation 5.4

If you cannot choose a concise name that expresses what the function does, it's possible that your function is attempting to perform too many diverse tasks. It's usually best to break such a function into smaller functions—this is sometimes called decomposition.

Function Definitions

- Each program we've presented has consisted of a function called `main` that called standard library functions to accomplish its tasks.
- We now consider how to write **custom functions**.
- Consider a program that uses a function `square` to calculate and print the squares of the integers from 1 to 10.

Function Definitions (cont'd)

```
1 // Creating and using a programmer-defined function.
2 #include <stdio.h>
3
4 int square(int y); // function prototype
5
6 int main(void)
7 {
8     // loop 10 times and calculate and output square of x each time
9     for (int x = 1; x <= 10; ++x) {
10         printf("%d ", square(x)); // function call
11     }
12
13     puts("");
14
15     return 0;
16 }
17
18 // square function definition returns the square of its parameter
19 int square(int y) // y is a copy of the argument to the function
20 {
21     return y * y; // returns the square of y as an int
22 }
```

1	4	9	16	25	36	49	64	81	100
---	---	---	----	----	----	----	----	----	-----

Function Definitions (cont'd)

- Calling Function `square`

- Function `square` is **invoked** or **called** in main within the `printf` statement

```
printf("%d ", square(x)); // function call
```

- Function `square` **receives a copy of the argument x's value in the parameter y.**

- Then `square` calculates $y * y$.

- The **result** is passed back **returned** to function `printf` in main **where `square` was invoked**, and `printf` displays the result.

- This process is repeated 10 times using the **for** statement.

Function Definitions (cont'd)

- `square` Function Definition

- The `definition of function` `square` shows that `square` `expects an integer parameter` `y`.
- The `keyword` `int` `preceding the function name` indicates that `square` `returns an integer` result.
- The `return` statement in `square` `passes the value` of the `expression` `y * y` (that is, the result of the calculation) `back to the calling function`.

Function Definitions (cont'd)

- square Function prototype

```
int square(int y); // function prototype
```

is a function prototype, a.k.a. function declaration.

- The `int` in parentheses informs the compiler that square expects to receive an integer value from the caller.
- The `int` to the left of the function name square informs the compiler that square returns an integer result to the caller.
- The compiler compares the calls to square to the function prototype to ensure:
 - ◆ the number of arguments is correct,
 - ◆ the arguments are of the correct types,
 - ◆ the arguments types are in the correct order, and
 - ◆ the return type is consistent with the context where the function is called.

Function Definitions (cont'd)

- Format of a **Function Definition**

- The format of a function definition is:

```
return-value-type function-name(parameter-list)
{
    statements
}
```

- The **function-name** is any valid identifier.
- The **return-value-type** is the data type of the result returned to the caller.
 - ◆ Void indicates that a function does not return a value.
- Together, the **return-value-type**, **function-name** and **parameter-list** are sometimes referred to as the **function header**.
- The **parameter-list** is a **comma-separated list** that specifies the parameters received by the function when it's called.
 - ◆ If a function does not receive any values, parameter-list is void.
 - ◆ A **type must be listed explicitly** for each parameter.

Function Definitions (cont'd)



Error-Prevention Tip 5.2

Check that your functions that are supposed to return values do so. Check that your functions that are not supposed to return values do not.

- ```
int function-name(parameter-list)
{
 statements;
 return expression;
}
```
- ```
void function-name(parameter-list)
{
    statements;
    return; //optional
}
```


Function Definitions (cont'd)



Common Programming Error 5.1

Specifying function parameters of the same type as `double x, y` instead of `double x, double y` results in a compilation error.

- ```
double function-name(double x, double y)
{
 statements;
 return expression;
}
```
- ```
double function-name(double x, y) // syntax error
{
    statements;
    return expression;
}
```

Function Definitions (cont'd)



Common Programming Error 5.2

Placing a semicolon after the right parenthesis enclosing the parameter list of a function definition is a syntax error.

- ```
double function-name(double x, double y);
{
 statements;
 return expression;
}
```

# Function Definitions (cont'd)



## Common Programming Error 5.3

*Redefining a parameter as a local variable in a function is a compilation error.*

```
• double function-name(double x, double y)
{
 double x; // syntax error
 statements;
 return expression;
}
```



## Good Programming Practice 5.2

*Although it's not incorrect to do so, do not use the same names for a function's arguments and the corresponding parameters in the function definition. This helps avoid ambiguity.*

```
• double z = function-name(x, y);
```

# Function Definitions (cont'd)

## ■ Function body

- ◆ The statements within braces form the **function body**, which is also referred to as a **block**.
- ◆ Variables can be declared in any block, and blocks can be nested.
- ◆ Functions cannot be nested.

# Function Definitions (cont'd)



## Common Programming Error 5.4

*Defining a function inside another function is a syntax error.*

```
• type function-name1(parameter-list1)
{
 type function-name1(parameter-list1)
 {
 statements;
 return expression;
 }
 statements;
 return expression;
}
```

# Function Definitions (cont'd)



## Good Programming Practice 5.3

*Choosing meaningful function names and meaningful parameter names makes programs more readable and helps avoid excessive use of comments.*

```
• double velocity(double offset, double time)
{
 if(time > 0) {
 return offset / time;
 }
 else {
 return -1;
 }
}
```



## Software Engineering Observation 5.5

*Small functions promote software reusability.*

# Function Definitions (cont'd)



## Software Engineering Observation 5.6

*Programs should be written as collections of small functions. This makes programs easier to write, debug, maintain and modify.*

```
● int main(void)
{
 double speed, finalPosition, duration;

 input(finalPosition, duration);
 speed = velocity(finalPosition, duration);

 printf("The velocity is %f.\n", speed);

 return 0;
}
```

# Function Definitions (cont'd)



## Software Engineering Observation 5.7

*A function requiring a large number of parameters may be performing too many tasks. Consider dividing the function into smaller functions that perform the separate tasks. The function header should fit on one line if possible.*

```
● double velocity(double offset, double time)
{
 puts("Enter the final position and duration:");
 if (scanf("%lf %lf", &offset, &time) == 2) {
 if(time > 0) {
 return offset / time;
 }
 else {
 return -1;
 }
 }
 else {
 puts("Ends with incorrect inputs.");
 }
}
```



# Function Definitions (cont'd)



## Software Engineering Observation 5.8

*The function prototype, function header and function calls should all agree in the number, type, and order of arguments and parameters, and in the type of return value.*

- `double velocity(double offset, unsigned int time);`
- ```
double velocity(double offset, unsigned int time)
{
    if(time > 0)
    {
        return offset / time;
    }
    else
    {
        return -1;
    }
}
```
- ```
double finalPosition = 5;
unsigned int duration = 10;
double speed = velocity(finalPosition, duration);
```

# Function Definitions (cont'd)

- Returning Control from a Function
  - There are three ways to return control from a called function to the point at which a function was invoked.
  - If the function **does not return** a result, control is returned simply when the **function-ending right brace is reached**, or by executing the statement `return;`
  - If the function **does return** a result, the statement `return expression;` returns the value of expression to the caller.

# Function Definitions (cont'd)

- `main`'s Return Type

- Notice that `main` has an `int` return type.
- The return value of `main` is used to indicate whether the program executed correctly.
- In earlier versions of C, we'd explicitly place `return 0;` at the end of `main`—0 indicates that a program ran successfully.
- The C standard indicates that `main` implicitly returns 0 if you to omit the preceding statement—as we've done throughout this book.

# Function Definitions (cont'd)

- You can **explicitly return non-zero** values from main to **indicate** that a **problem** occurred during your program's execution.
- For information on how to report a program failure, see the documentation for your particular operating-system environment.

# Function Definitions (cont'd)

- Function `maximum`

- Our second example uses a programmer-defined function `maximum` to determine and return the largest of three integers.
- The integers are input with `scanf`.
- Next, they're passed to `maximum`, which determines the largest integer.
- This value is returned to `main` by the `return` statement in `maximum`.

# Function Definitions (cont'd)

```
1 // Finding the maximum of three integers.
2 #include <stdio.h>
3
4 int maximum(int x, int y, int z); // function prototype
5
6 int main(void)
7 {
8 int number1; // first integer entered by the user
9 int number2; // second integer entered by the user
10 int number3; // third integer entered by the user
11
12 printf("%s", "Enter three integers: ");
13 scanf("%d%d%d", &number1, &number2, &number3);
14
15 // number1, number2 and number3 are arguments
16 // to the maximum function call
17 printf("Maximum is: %d\n", maximum(number1, number2, number3));
18
19 return 0;
20 }
```

# Function Definitions (cont'd)

```
22 // Function maximum definition
23 // x, y and z are parameters
24 int maximum(int x, int y, int z)
25 {
26 int max = x; // assume x is largest
27
28 if (y > max) { // if y is larger than max,
29 max = y; // assign y to max
30 }
31
32 if (z > max) { // if z is larger than max,
33 max = z; // assign z to max
34 }
35
36 return max; // max is largest
37 }
```

Enter three integers: 22 85 17  
Maximum is: 85

Enter three integers: 47 32 14  
Maximum is: 47

Enter three integers: 35 8 79  
Maximum is: 79

# Function Prototypes: A Deeper Look

- An important feature of C is the function prototype.
- This feature was borrowed from C++.
- The compiler uses function prototypes to validate function calls.
- Early versions of C did not perform this kind of checking, so it was possible to call functions improperly without the compiler detecting the errors.
- Such calls could result in fatal execution-time errors or nonfatal errors that caused subtle, difficult-to-detect problems.



## Good Programming Practice 5.4

*Include function prototypes for all functions to take advantage of C's type-checking capabilities. Use `#include` preprocessor directives to obtain function prototypes for the standard library functions from the headers for the appropriate libraries, or to obtain headers containing function prototypes for functions developed by you and/or your group members.*



# Function Prototypes: A Deeper Look (cont'd)

- The function prototype for `maximum` is

```
int maximum(int x, int y, int z); // function prototype
```

- It states that `maximum` takes **three arguments of type `int`** and returns **a result of type `int`**.
- Notice that the function prototype is **the same as the first line** of `maximum`'s **function definition**.



## Good Programming Practice 5.5

*Include parameter names in function prototypes for documentation purposes. The compiler ignores these names, so the prototype `int maximum(int, int, int);` is valid.*



## Common Programming Error 5.5

*Forgetting the semicolon at the end of a function prototype is a syntax error.*

# Function Prototypes: A Deeper Look (cont'd)

## ● Compilation Errors

- A function **call** that does **not match** the function **prototype** is a compilation error.
- An error is also generated if the function **prototype** and the function **definition disagree**.
- For example, if the function prototype had been written

```
void maximum(int x, int y, int z);
```

the compiler would generate an error because the `void` return type in the function prototype would differ from the `int` return type in the function header.

# Function Prototypes: A Deeper Look (cont'd)

- Argument Coercion and “Usual Arithmetic Conversion Rules”
  - Another important feature of function prototypes is the coercion of arguments, i.e., the forcing of arguments to the appropriate type.
  - For example, the math library function `sqrt` can be called with an integer argument even though the function prototype in `<math.h>` specifies a double parameter, and the function will still work correctly.
  - The statement

```
printf("%.3f\n", sqrt(4));
```

correctly evaluates `sqrt(4)` and prints the value 2.000.

# Function Prototypes: A Deeper Look (cont'd)

- The function prototype causes the compiler to convert a copy of the integer value 4 to the double value 4.0 before the copy is passed to `sqrt`.
- In general, argument values that do not correspond precisely to the parameter types in the function prototype are converted to the proper type before the function is called.
- These conversions can lead to incorrect results if C's usual arithmetic conversion rules are not followed.
- These rules specify how values can be converted to other types without losing data.

# Function Prototypes: A Deeper Look (cont'd)

- In our `sqrt` example above, an `int` is automatically converted to a `double` without changing its value.
- However, a `double` converted to an `int` truncates the fractional part of the `double` value, thus changing the original value.
- Converting large integer types to small integer types (e.g., `long` to `short`) may also result in changed values.

# Function Prototypes: A Deeper Look (cont'd)

- The **usual arithmetic conversion rules** automatically apply to expressions containing values of two or more data types (also referred to as **mixed-type expressions**) and are handled for you by the compiler.
- In a mixed-type expression, the compiler makes a temporary copy of the value that needs to be converted then **converts the copy to the “highest” type in the expression**—this is known as **promotion**.

# Function Prototypes: A Deeper Look (cont'd)

- The usual arithmetic conversion rules for a mixed-type expression **containing at least one floating-point value** are:
  - ◆ If one of the values is a long double, the other is converted to a long double.
  - ◆ If one of the values is a double, the other is converted to a double.
  - ◆ If one of the values is a float, the other is converted to a float.

# Function Prototypes: A Deeper Look (cont'd)

- If the mixed-type expression contains only integer types, then the usual arithmetic conversions specify a set of integer promotion rules.
- In most cases, the integer types lower are converted to types higher.
- Section 6.3.1 of the C standard document specifies the complete details of arithmetic operands and the usual arithmetic conversion rules.



# Function Prototypes: A Deeper Look (cont'd)

- Arithmetic data types and their conversion specifiers

| Data type                   | printf conversion specification | scanf conversion specification |
|-----------------------------|---------------------------------|--------------------------------|
| <i>Floating-point types</i> |                                 |                                |
| long double                 | %Lf                             | %Lf                            |
| double                      | %f                              | %lf                            |
| float                       | %f                              | %f                             |
| <i>Integer types</i>        |                                 |                                |
| unsigned long long int      | %llu                            | %llu                           |
| long long int               | %lld                            | %lld                           |
| unsigned long int           | %lu                             | %lu                            |
| long int                    | %ld                             | %ld                            |
| unsigned int                | %u                              | %u                             |
| int                         | %d                              | %d                             |
| unsigned short              | %hu                             | %hu                            |
| short                       | %hd                             | %hd                            |
| char                        | %c                              | %c                             |

# Function Prototypes: A Deeper Look (cont'd)

- A value can be converted to a lower type only by explicitly assigning the value to a variable of lower type, or by using a cast operator.
- Function argument values are converted to the parameter types in a function prototype as if they were being assigned directly to variables of those types.
- If our `square` function that uses an integer parameter is called with a floating-point argument, the argument is converted to `int` (a lower type), and `square` usually returns an incorrect value.
- For example, `square(4.5)` returns 16, not 20.25.



## Common Programming Error 5.6

*Converting from a higher data type in the promotion hierarchy to a lower type can change the data value. Many compilers issue warnings in such cases.*

# Function Prototypes: A Deeper Look.

- If there is **no function prototype** for a function, the compiler forms its own function prototype using the **first occurrence of the function**—either the function definition or a call to the function.
- This typically leads to warnings or errors, depending on the compiler.



## Error-Prevention Tip 5.3

*Always include function prototypes for the functions you define or use in your program to help prevent compilation errors and warnings.*



## Software Engineering Observation 5.9

*A function prototype placed outside any function definition applies to all calls to the function appearing after the function prototype in the file. A function prototype placed in a function body applies only to calls made in that function.*

# Headers

- Each standard library has a corresponding **header** containing the **function prototypes** for all the functions in that library and definitions of various **data types** and **constants** needed by those functions.

# Headers (cont'd)

| Header                        | Explanation                                                                                                                                                                                                                                                                                                                |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>&lt;assert.h&gt;</code> | Contains information for adding diagnostics that aid program debugging.                                                                                                                                                                                                                                                    |
| <code>&lt;ctype.h&gt;</code>  | Contains function prototypes for functions that test characters for certain properties, and function prototypes for functions that can be used to convert lowercase letters to uppercase letters and vice versa.                                                                                                           |
| <code>&lt;errno.h&gt;</code>  | Defines macros that are useful for reporting error conditions.                                                                                                                                                                                                                                                             |
| <code>&lt;float.h&gt;</code>  | Contains the floating-point size limits of the system.                                                                                                                                                                                                                                                                     |
| <code>&lt;limits.h&gt;</code> | Contains the integral size limits of the system.                                                                                                                                                                                                                                                                           |
| <code>&lt;locale.h&gt;</code> | Contains function prototypes and other information that enables a program to be modified for the current locale on which it's running. The notion of locale enables the computer system to handle different conventions for expressing data such as dates, times, currency amounts and large numbers throughout the world. |
| <code>&lt;math.h&gt;</code>   | Contains function prototypes for math library functions.                                                                                                                                                                                                                                                                   |
| <code>&lt;setjmp.h&gt;</code> | Contains function prototypes for functions that allow bypassing of the usual function call and return sequence.                                                                                                                                                                                                            |
| <code>&lt;signal.h&gt;</code> | Contains function prototypes and macros to handle various conditions that may arise during program execution.                                                                                                                                                                                                              |

# Headers (cont'd)

| Header                        | Explanation                                                                                                                                         |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>&lt;stdarg.h&gt;</code> | Defines macros for dealing with a list of arguments to a function whose number and types are unknown.                                               |
| <code>&lt;stddef.h&gt;</code> | Contains common type definitions used by C for performing calculations.                                                                             |
| <code>&lt;stdio.h&gt;</code>  | Contains function prototypes for the standard input/output library functions, and information used by them.                                         |
| <code>&lt;stdlib.h&gt;</code> | Contains function prototypes for conversions of numbers to text and text to numbers, memory allocation, random numbers and other utility functions. |
| <code>&lt;string.h&gt;</code> | Contains function prototypes for string-processing functions.                                                                                       |
| <code>&lt;time.h&gt;</code>   | Contains function prototypes and types for manipulating the time and date.                                                                          |

# Headers.

- You can create custom headers.
- Programmer-defined headers should also use the .h filename extension.
- A programmer-defined header can be included by using the `#include` preprocessor directive.
- For example, if the prototype for our square function was located in the header `square.h`, we'd include that header in our program by using the following directive at the top of the program:

```
#include "square.h"
```

# Passing Arguments By Value and By Reference

- In many programming languages, there are two ways to pass arguments—pass-by-value (call-by-value) and pass-by-reference (call-by-reference).
- When arguments are **passed by value**, a copy of the **argument's value** is made and **passed** to the called function.
- Changes to the copy do **not** affect an original variable's value in the caller.
- When an argument is **passed by reference**, the caller **allows** the called function to **modify** the original variable's value.
- **Pass-by-value** should be used whenever the called function does **not need to modify the value** of the caller's original variable.



# Passing Arguments By Value and By Reference.

- This prevents the accidental side effects (variable modifications) that so greatly hinder the development of correct and reliable software systems.
- **Pass-by-reference** should be used only with **trusted called functions** that need to modify the original variable.
- In C, all arguments are passed by value.
- It is possible to achieve pass-by-reference in C. We will show in later chapters.

# Random Number Generation

- We now take a brief and, hopefully, entertaining diversion into simulation and game playing.
- The element of chance can be introduced into computer applications by using the C standard library function `rand` from the `<stdlib.h>` header.
- Consider the following statement:  

```
i = rand();
```
- The `rand` function generates an integer between 0 and `RAND_MAX` (a **symbolic constant** defined in the `<stdlib.h>` header).

# Random Number Generation (cont'd)

- Standard C states that the value of `RAND_MAX` must be at least 32767, which is the maximum value for a two-byte (i.e., 16-bit) integer.
- The programs in this section were tested on Microsoft Visual C++ with a maximum `RAND_MAX` value of 32767 and on GNU gcc and Xcode LLVM with a `RAND_MAX` value of 2147483647.
- If `rand` truly produces integers at random, every number between 0 and `RAND_MAX` has an equal chance (or probability) of being chosen each time `rand` is called.
- The range of values produced directly by `rand` is often different from what's needed in a specific application.

# Random Number Generation (cont'd)

- For example, a program that simulates coin tossing might require only 0 for “heads” and 1 for “tails.”
- A dice-rolling program that simulates a six-sided die would require random integers from 1 to 6.
- Rolling a Six-Sided Die
  - To demonstrate `rand`, let's develop a program to simulate 20 rolls of a six-sided die and print the value of each roll.
  - The function prototype for function `rand` is in `<stdlib.h>`.
  - We use the remainder operator (%) in conjunction with `rand` as follows  
`rand() % 6`  
to produce integers in the range 0 to 5.

# Random Number Generation (cont'd)

- This is called **scaling**.
- The number **6** is called the **scaling factor**.
- We then **shift the range** of numbers produced **by adding 1** to our previous result.
- The results are in the range 1 to 6—the actual random values chosen might vary by compiler.

# Random Number Generation (cont'd)

```
1 // Shifted, scaled random integers produced by 1 + rand() % 6.
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 int main(void)
6 {
7 // loop 20 times
8 for (unsigned int i = 1; i <= 20; ++i) {
9
10 // pick random number from 1 to 6 and output it
11 printf("%10d", 1 + (rand() % 6));
12
13 // if counter is divisible by 5, begin new line of output
14 if (i % 5 == 0) {
15 puts("");
16 }
17 }
18
19 return 0;
20 }
```

|   |   |   |   |   |
|---|---|---|---|---|
| 6 | 6 | 5 | 5 | 6 |
| 5 | 1 | 1 | 5 | 3 |
| 6 | 6 | 2 | 4 | 2 |
| 6 | 2 | 3 | 4 | 1 |

# Random Number Generation (cont'd)

- Rolling a Six-Sided Die 60,000,000 Times
  - To show that these numbers **occur approximately with equal likelihood**, let's simulate 60,000,000 rolls of a die with the program.
  - Each integer from 1 to 6 should appear **approximately 10,000,000** times.
  - As the program output shows, by scaling and shifting we've used the `rand` function to realistically simulate the rolling of a six-sided die.

# Random Number Generation (cont'd)

```
1 // Rolling a six-sided die 60,000,000 times.
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 int main(void)
6 {
7 unsigned int frequency1 = 0; // rolled 1 counter
8 unsigned int frequency2 = 0; // rolled 2 counter
9 unsigned int frequency3 = 0; // rolled 3 counter
10 unsigned int frequency4 = 0; // rolled 4 counter
11 unsigned int frequency5 = 0; // rolled 5 counter
12 unsigned int frequency6 = 0; // rolled 6 counter
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47 // display results in tabular format
48 printf("%s%13s\n", "Face", "Frequency");
49 printf(" 1%13u\n", frequency1);
50 printf(" 2%13u\n", frequency2);
51 printf(" 3%13u\n", frequency3);
52 printf(" 4%13u\n", frequency4);
53 printf(" 5%13u\n", frequency5);
54 printf(" 6%13u\n", frequency6);
55
56 return 0;
57 }
```



R

'd)

```
14 // loop 60000000 times and summarize results
15 for (unsigned int roll = 1; roll <= 60000000; ++roll) {
16 int face = 1 + rand() % 6; // random number from 1 to 6
17
18 // determine face value and increment appropriate counter
19 switch (face) {
20
21 case 1: // rolled 1
22 ++frequency1;
23 break;
24
25 case 2: // rolled 2
26 ++frequency2;
27 break;
28
29 case 3: // rolled 3
30 ++frequency3;
31 break;
32
33 case 4: // rolled 4
34 ++frequency4;
35 break;
36
37 case 5: // rolled 5
38 ++frequency5;
39 break;
40
41 case 6: // rolled 6
42 ++frequency6;
43 break; // optional
44 }
45 }
```

| Face | Frequency |
|------|-----------|
| 1    | 9999294   |
| 2    | 10002929  |
| 3    | 9995360   |
| 4    | 10000409  |
| 5    | 10005206  |
| 6    | 9996802   |

# Random Number Generation (cont'd)

- Note the use of the `%s` conversion specifier to print the character `strings` "Face" and "Frequency" as column headers.
- After we study `arrays` in Chapter 6, we'll show how to replace this switch statement elegantly with a `single-line statement`.

# Random Number Generation (cont'd)

- Randomizing the Random Number Generator
  - Executing the program again produces exactly the same sequence of values
  - How can these be **random numbers**? Ironically, this repeatability is an important characteristic of function `rand`.

# Random Number Generation (cont'd)

- When **debugging** a program, this **repeatability** is essential for proving that corrections to a program work properly.
- Function `rand` actually generates **pseudorandom numbers**.
- Calling `rand` repeatedly produces a sequence of numbers that appears to be random.
- However, the sequence repeats itself each time the program is executed.
- Once a program has been thoroughly debugged, it can be conditioned to produce a different sequence of random numbers for each execution.

# Random Number Generation (cont'd)

- This is called `randomizing` and is accomplished with the standard library function `srand`.
- Function `srand` takes an `unsigned integer argument` and seeds function `rand` to produce a different sequence of random numbers for each execution of the program.
- The function prototype for `srand` is found in `<stdlib.h>`.

# Random Number Generation (cont'd)

```
1 // Randomizing die-rolling program.
2 #include <stdlib.h>
3 #include <stdio.h>
4
5 int main(void)
6 {
7 unsigned int seed; // number used to seed the random number generator
8
9 printf("%s", "Enter seed: ");
10 scanf("%u", &seed); // note %u for unsigned int
11
12 srand(seed); // seed the random number generator
13
14 // loop 10 times
15 for (unsigned int i = 1; i <= 10; ++i) {
16
17 // pick a random number from 1 to 6 and output it
18 printf("%10d", 1 + (rand() % 6));
19
20 // if counter is divisible by 5, begin a new line of output
21 if (i % 5 == 0) {
22 puts("");
23 }
24 }
25
26 return 0;
27 }
```

|                |   |   |   |   |
|----------------|---|---|---|---|
| Enter seed: 67 |   |   |   |   |
| 6              | 1 | 4 | 6 | 2 |
| 1              | 6 | 1 | 6 | 4 |

|                 |   |   |   |   |
|-----------------|---|---|---|---|
| Enter seed: 867 |   |   |   |   |
| 2               | 4 | 6 | 1 | 6 |
| 1               | 1 | 3 | 6 | 2 |

|                |   |   |   |   |
|----------------|---|---|---|---|
| Enter seed: 67 |   |   |   |   |
| 6              | 1 | 4 | 6 | 2 |
| 1              | 6 | 1 | 6 | 4 |

# Random Number Generation (cont'd)

- Notice that a different sequence of random numbers is obtained each time the program is run, provided that a different seed is supplied.
- To randomize without entering a seed each time, use a statement like

```
srand(time(NULL));
```
- This causes the computer to read its clock to obtain the value for the seed automatically.
- Function `time` returns the number of seconds that have passed since midnight on January 1, 1970.
- This value is converted to an unsigned integer and used as the seed to the random number generator.
- The function prototype for `time` is in `<time.h>`.

# Random Number Generation (cont'd)

- Generalized Scaling and Shifting of Random Numbers
  - The values produced directly by rand are always in the range:  
$$0 \leq \text{rand}() \leq \text{RAND\_MAX}$$
  - As you know, the following statement simulates rolling a six-sided die:  
`face = 1 + rand() % 6;`
  - This statement always assigns an integer value at random to the variable face in the range  $1 \leq \text{face} \leq 6$ .
  - The width of this range (i.e., the number of consecutive integers in the range) is 6 and the starting number in the range is 1.



# Random Number Generation.

- Referring to the preceding statement, we see that the width of the range is determined by the number used to scale rand with the remainder operator (i.e., 6), and the starting number of the range is equal to the number (i.e., 1) that's added to rand % 6.
- We can generalize this result as follows
$$n = a + \text{rand}() \% b;$$
- where **a is the shifting value** (which is equal to the first number in the desired range of consecutive integers) and **b is the scaling factor** (which is equal to the width of the desired range of consecutive integers).

# Rounding Numbers

- The function `floor(x)` returns the largest integer less than `x`.
- We can use the function `floor` to round numbers in various ways:
  - Round to integer  
`y = floor (x + 0.5);`
  - Round to the tens position  
`y = floor (x / 10 + 0.5) * 10`
  - Round to the hundreds position  
`y = floor (x / 100 + 0.5) * 100`

# Example: A Game of Chance; Introducing `enum`

- One of the most popular games of chance is a dice game known as “`craps`.” The rules of the game are simple.
  - A player rolls two dice. Each die has six faces. These faces contain 1, 2, 3, 4, 5, and 6 spots. After the dice have come to rest, the sum of the spots on the two upward faces is calculated. If the `sum is 7 or 11` on the first throw, the player `wins`. If the `sum is 2, 3, or 12` on the first throw (called “`craps`”), the player `loses` (i.e., the “house” wins). If the `sum is 4, 5, 6, 8, 9, or 10` on the first throw, then that sum becomes the player’s “point.” To win, you must `continue rolling the dice until you “make your point.”` The player `loses by rolling a 7` before making the point.

# Example: A Game of Chance; Introducing `enum` (cont'd)

```
1 // Simulating the game of craps.
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <time.h> // contains prototype for function time
5
6 // enumeration constants represent game status
7 enum Status { CONTINUE, WON, LOST };
8
9 int rollDice(void); // function prototype
10
11 int main(void)
12 {
13 // randomize random number generator using current time
14 srand(time(NULL));
15
16 int myPoint; // player must make this point to win
17 enum Status gameStatus; // can contain CONTINUE, WON, or LOST
18 int sum = rollDice(); // first roll of the dice
```

# Example: A Game of Chance; Introducing `enum` (cont'd)

```
70 // roll dice, calculate sum and display results
71 int rollDice(void)
72 {
73 int die1 = 1 + (rand() % 6); // pick random die1 value
74 int die2 = 1 + (rand() % 6); // pick random die2 value
75
76 // display results of this roll
77 printf("Player rolled %d + %d = %d\n", die1, die2, die1 + die2);
78 return die1 + die2; // return sum of dice
79 }
```

# Example: A Game of Chance; Introducing `enum` (cont'd)

```
20 // determine game status based on sum of dice
21 switch(sum) {
22
23 // win on first roll
24 case 7: // 7 is a winner
25 case 11: // 11 is a winner
26 gameStatus = WON;
27 break;
28
29 // lose on first roll
30 case 2: // 2 is a loser
31 case 3: // 3 is a loser
32 case 12: // 12 is a loser
33 gameStatus = LOST;
34 break;
35
36 // remember point
37 default:
38 gameStatus = CONTINUE; // player should keep rolling
39 myPoint = sum; // remember the point
40 printf("Point is %d\n", myPoint);
41 break; // optional
42 }
```

# Example: A Game of Chance; Introducing enum (cont'd)

```
44 // while game not complete
45 while (CONTINUE == gameStatus) { // player should keep rolling
46 sum = rollDice(); // roll dice again
47
48 // determine game status
49 if (sum == myPoint) { // win by making point
50 gameStatus = WON;
51 }
52 else {
53 if (7 == sum) { // lose by rolling 7
54 gameStatus = LOST;
55 }
56 }
57 }
58
59 // display won or lost message
60 if (WON == gameStatus) { // did player win?
61 puts("Player wins");
62 }
63 else { // player lost
64 puts("Player loses");
65 }
66
67 return 0;
```

Player rolled 5 + 6 = 11  
Player wins

Player rolled 4 + 1 = 5  
Point is 5  
Player rolled 6 + 2 = 8  
Player rolled 2 + 1 = 3  
Player rolled 3 + 2 = 5  
Player wins

Player rolled 6 + 4 = 10  
Point is 10  
Player rolled 3 + 4 = 7  
Player loses

Player rolled 1 + 1 = 2  
Player loses

# Example: A Introducing

```
70 // roll dice, calculate sum and display results
71 int rollDice(void)
72 {
73 int die1 = 1 + (rand() % 6); // pick random die1 value
74 int die2 = 1 + (rand() % 6); // pick random die2 value
75
76 // display results of this roll
77 printf("Player rolled %d + %d = %d\n", die1, die2, die1 + die2);
78 return die1 + die2; // return sum of dice
79 }
```

- In the rules of the game, notice that the player must **roll two dice** on the first roll, and must do so later on all subsequent rolls.
- We define a function `rollDice` to roll the dice and compute and print their sum.
- Function `rollDice` is defined once, but it's called from two places in the program.
- Interestingly, `rollDice` takes **no arguments**, so we've indicated `void` in the parameter list
- Function `rollDice` does **return the sum** of the two dice, so a return type of `int` is indicated in its function header and in its function prototype.



# Example: A Game of Chance; Introducing `enum` (cont'd)

- The player may win or lose on the first roll, or may win or lose on any subsequent roll.
- Variable `gameStatus`, defined to be of a new type—`enum Status`—stores the current status.
- An **enumeration**, introduced by the keyword `enum`, is a set of integer constants represented by identifiers.
- Enumeration constants help make programs easier to read.
- Values in an `enum` start with 0 and are incremented by 1.

# Example: A Game of Chance; Introducing `enum` (cont'd)

## • Enumerations

```
enum Status { CONTINUE, WON, LOST };
```

- The **constant** `CONTINUE` has the value 0, `WON` has the value 1 and `LOST` has the value 2.
- It's also possible to assign an integer value to each identifier in an `enum` (see Chapter 10).
- The **identifiers** in an enumeration **must be unique**, but the **values may be duplicated**.



### Common Programming Error 5.7

*Assigning a value to an enumeration constant after it has been defined is a syntax error.*



### Good Programming Practice 5.6

*Use only uppercase letters in the names of enumeration constants to make these constants stand out in a program and to indicate that enumeration constants are not variables.*

# Example: A Game of Chance; Introducing `enum` (cont'd)

- When the game is **won**, either on the first roll or on a subsequent roll, `gameStatus` is set to `WON`.  
`gameStatus = WON;`
- When the game is **lost**, either on the first roll or on a subsequent roll, `gameStatus` is set to `LOST`.  
`gameStatus = LOST;`
- **Otherwise** `gameStatus` is set to `CONTINUE` and the game continues.  
`gameStatus = CONTINUE;`  
`while (CONTINUE == gameStatus) ...`

# Example: A Game of Chance; Introducing `enum` (cont'd)

- Game Ends on First Roll

- After the first roll, if the game is over, the while statement is skipped because `gameStatus` is not `CONTINUE`.

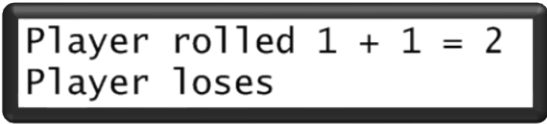
```
while (CONTINUE == gameStatus)
```

- The program proceeds to the `if...else` statement, which prints "Player wins" if `gameStatus` is `WON` and "Player loses" otherwise.

```
if (WON == gameStatus) { // did player win?
 puts("Player wins");
}
else { // player lost
 puts("Player loses");
}
```



```
Player rolled 5 + 6 = 11
Player wins
```



```
Player rolled 1 + 1 = 2
Player loses
```

# Example: A Game of Chance; Introducing `enum` (cont'd)

- Game Ends on a Subsequent Roll

- After the first roll, if the game is not over, then `sum` is saved in `myPoint`.

```
default:
 gameStatus = CONTINUE; // player should keep rolling
 myPoint = sum; // remember the point
 printf("Point is %d\n", myPoint);
 break; // optional
```

- Execution proceeds with the `while` statement because `gameStatus` is `CONTINUE`.
- Each time through the `while`, `rollDice` is called to produce a new `sum`.

```
sum = rollDice(); // roll dice again
```

# Example: A Game of Chance; Introducing `enum` (cont'd)

- If `sum` matches `myPoint`, `gameStatus` is set to `WON` to indicate that the player won, the while-test fails, the `if...else` statement prints "Player wins" and execution terminates.

```
// determine game status
if (sum == myPoint) { // win by making point
 gameStatus = WON;
}
```

- If `sum` is equal to 7, `gameStatus` is set to `LOST` to indicate that the player lost, the while-test fails, the `if...else` statement prints "Player loses" and execution terminates.

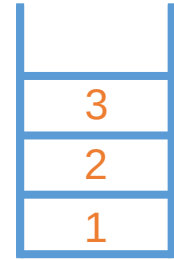
```
if (7 == sum) { // lose by rolling 7
 gameStatus = LOST;
}
```

# Example: A Game of Chance; Introducing `enum`.

- Control Architecture

- Note the program's interesting control architecture.
- We've used two functions—`main` and `rollDice`—and the `switch`, `while`, `nested if...else` and `nested if` statements.
- In the exercises, we'll investigate various interesting characteristics of the game of craps.

# Function Call Stack and Stack Frames



- To understand how C performs function calls, we first need to consider a data structure (i.e., collection of related data items) known as a **stack**.
- Think of a stack as analogous to a pile of dishes.
- When a dish is placed on the pile, it's normally **placed at the top** (referred to as **pushing** the dish onto the stack).
- Similarly, when a dish is removed from the pile, it's normally **removed from the top** (referred to as popping the dish off the stack).
- Stacks are known as **last-in, first-out (LIFO) data structures**—the last item pushed (inserted) on the stack is the first item popped (removed) from the stack.



# Function Call Stack and Stack Frames (cont'd)

- An important mechanism for computer science students to understand is the **Function Call Stack** (sometimes referred to as the **program execution stack**).
- This data structure—working “behind the scenes”—**supports the function call/return mechanism**.
- It also supports the creation, maintenance and destruction of each **called function's local variables**.

# Function Call Stack and Stack Frames (cont'd)

- As each function is called, it may call other functions, which may call other functions—all before any function returns.
- Each function eventually must return control to the function that called it.
- So, we must keep track of the return addresses that each function needs to return control to the function that called it.
- The function call stack is the perfect data structure for handling this information.

# Function Call Stack and Stack Frames (cont'd)

- Each time a function calls another function, an entry is pushed onto the stack.
- This entry, called a stack frame, contains the return address that the called function needs in order to return to the calling function.
- If the called function returns, instead of calling another function before returning, the stack frame for the function call is popped, and control transfers to the return address in the popped stack frame.

# Function Call Stack and Stack Frames (cont'd)

- Each **called function** always **finds** the **information** it needs to return to its caller **at the top of the call stack**.
- And, if a function **makes a call** to another function, **a stack frame** for the new function call is simply **pushed onto the call stack**.
- Thus, the return address required by the newly called function to return to its caller is now located at the top of the stack.

# Function Call Stack and Stack Frames (cont'd)

- The stack frames have another important responsibility.
- Most functions have **automatic variables**—parameters and some or all of their local variables.
- Automatic variables need to exist while a function is executing.
- They need to **remain active** if the function makes **calls** to other functions.
- But when a called function **returns** to its caller, the called function's **automatic variables need to “go away.”**

# Function Call Stack and Stack Frames (cont'd)

- The called function's **stack frame** is a perfect place to reserve the memory **for automatic variables**.
- That stack frame exists only as long as the called function is active.
- When that **function returns**—and no longer needs its local automatic variables—its **stack frame is popped** from the stack, and those **local automatic variables are no longer known** to the program.

# Function Call Stack and Stack Frames (cont'd)

- Of course, the amount of memory in a computer is finite, so **only a certain amount of memory can be used to store stack frames** on the function call stack.
- If more function calls occur than can have their stack frames stored on the function call stack, a fatal error known as a **stack overflow** occurs.

# Function Call Stack and Stack Frames (cont'd)

- Function Call Stack in Action

- Now let's consider how the call stack supports the operation of a `square` function called by `main`

```
1 // Demonstrating the function call stack
2 // and stack frames using a function square.
3 #include <stdio.h>
4
5 int square(int); // prototype for function square
6
7 int main()
8 {
9 int a = 10; // value to square (local automatic variable in main)
10
11 printf("%d squared: %d\n", a, square(a)); // display a squared
12
13 return 0;
14 }
15
16 // returns the square of an integer
17 int square(int x) // x is a local variable
18 {
19 return x * x; // calculate square and return res
20 }
```

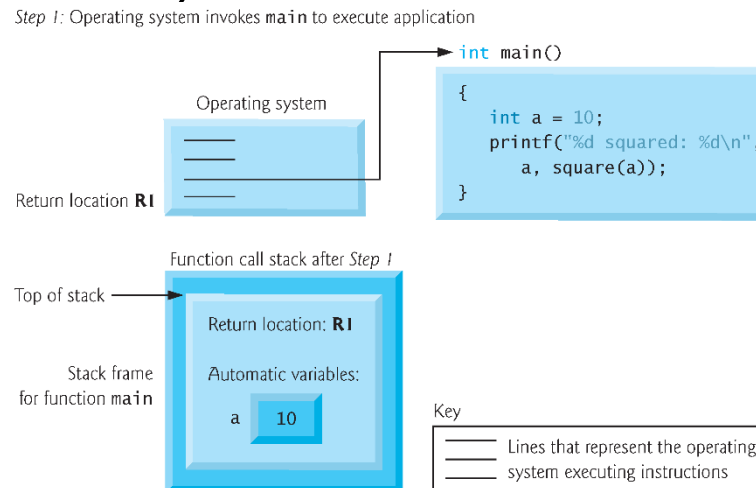
10 squared: 100



# Function Call Stack and Stack Frames (cont'd)

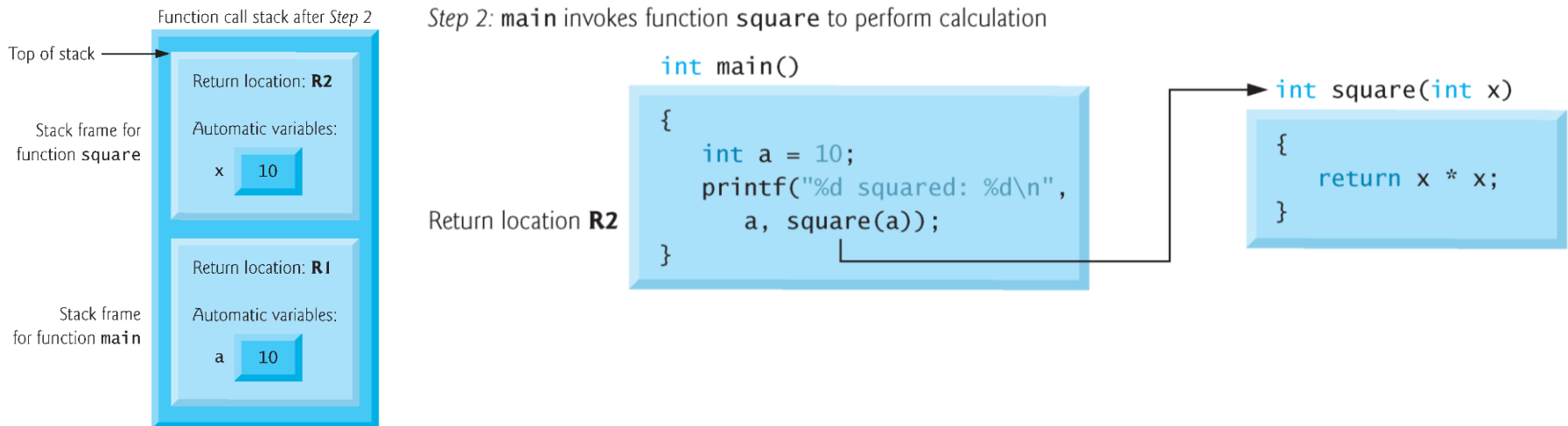
## ● Function Call Stack in Action

- First the operating system calls `main`—this pushes a stack frame onto the stack.
  - ◆ The stack frame tells `main` how to return to the operating system (i.e., transfer to return address `R1`) and contains the space for `main`'s automatic variable (i.e., `a`, which is initialized to 10).



# Function Call Stack and Stack Frames (cont'd)

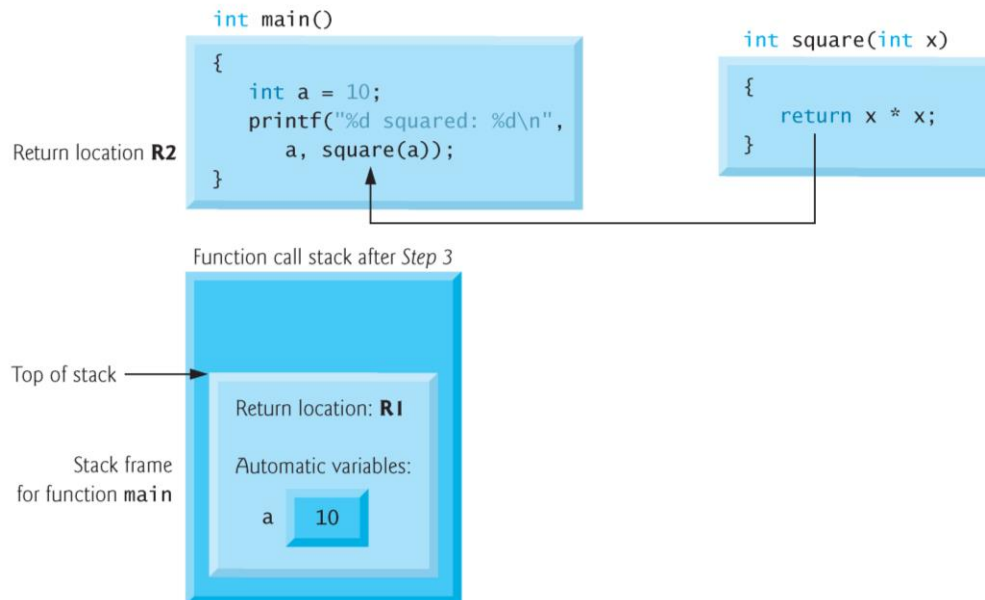
- Function main—before returning to the operating system—now calls function square
  - ◆ This causes a stack frame for square to be pushed onto The Function Call Stack
  - ◆ This stack frame contains the return address that square needs to return to main (i.e., R2) and the memory for square's automatic variable (i.e., x).



# Function Call Stack and Stack Frames (cont'd)

- After square calculates the square of its argument, it needs to return to main—and no longer needs the memory for its automatic variable x.
- So the stack is popped—giving square the return location in main (i.e., R2) and losing square's automatic variable.

Step 3: square returns its result to main



# Function Call Stack and Stack Frames (cont'd)

- Function main now displays the result of calling square
- Reaching the closing right brace of main causes its stack frame to be popped from the stack, gives main the address it needs to return to the operating system (i.e., R1) and causes the memory for main's automatic variable (i.e., a) to become unavailable.

# Storage Classes

- In Chapters 2–4, we used identifiers for variable names.
- The attributes of variables include name, type, size and value.
- In this chapter, we also use identifiers as names for user-defined functions.
- Actually, each identifier in a program has other attributes, including **storage class**, **storage duration**, **scope** and **linkage**.
- C provides the storage class specifiers: **auto**, **register**, **extern** and **static**.
- An identifier's **storage class** determines its **storage duration**, **scope** and **linkage**.
- An identifier's **storage duration** is the period during which the identifier exists in memory.

# Storage Classes (cont'd)

- Some exist briefly, some are repeatedly created and destroyed, and others exist for the program's entire execution.
- An identifier's **scope** is where the identifier can be referenced in a program.
- Some can be referenced throughout a program, others from only portions of a program.
- An identifier's **linkage** determines for a multiple-source-file program whether the identifier is known only in the current source file or in any source file with proper declarations (CH 14).

# Storage Classes (cont'd)

- The storage-class specifiers can be split into **automatic storage duration** and **static storage duration**.
- Keyword `auto` is used to declare variables of automatic storage duration.
- Variables with automatic storage duration are created when the block in which they're defined is entered; they **exist while the block is active**, and they're **destroyed when the block is exited**.

# Storage Classes (cont'd)

## ● Local Variables

- Only variables can have **automatic storage duration**.
- A function's local variables (those declared in the parameter list or function body) normally have automatic storage duration.
- Keyword `auto` **explicitly declares** variables of automatic storage duration.
- Local variables have automatic storage duration by default, so keyword `auto` is rarely used.
- For the remainder of the text, we'll refer to variables with automatic storage duration simply as **automatic variables**.



### **Performance Tip 5.1**

*Automatic storage is a means of conserving memory, because automatic variables exist only when they're needed. They're created when a function is entered and destroyed when the function is exited.*



# Storage Classes (cont'd)

- Static Storage Class

- Keywords `extern` and `static` are used in the declarations of identifiers for `variables and functions of static storage duration`.
- Identifiers of static storage duration `exist from the time at which the program begins execution until the program terminates`.
- For static variables, `storage is allocated and initialized only once, before the program begins execution`.
- For functions, the name of the function exists when the program begins execution.

# Storage Classes (cont'd)

- However, even though the variables and the function names exist from the start of program execution, this does not mean that these identifiers can be accessed throughout the program.
- Storage duration and scope (where a name can be used) are separate issues.
- There are several types of identifiers with static storage duration: external identifiers (such as global variables and function names) and local variables declared with the storage-class specifier `static`.



### Software Engineering Observation 5.10

*Defining a variable as global rather than local allows unintended side effects to occur when a function that does not need access to the variable accidentally or maliciously modifies it. In general, global variables should be avoided except in certain situations with unique performance requirements (as discussed in Chapter 14).*

## ■ Global Variables and Function Names

- ◆ Are of storage class `extern` by default
- ◆ Global variables are created by **placing variable declarations outside any function definition**, and they retain their values throughout the execution of the program.
- ◆ Global variables and functions **can be referenced by any function** that follows their declarations or definitions in the file.
- ◆ This is one reason for using **function prototypes**—when we include `stdio.h` in a program that calls `printf`, the function prototype is placed at the start of our file to make the name `printf` known to the rest of the file.



### Software Engineering Observation 5.11

*Variables used only in a particular function should be defined as local variables in that function rather than as external variables.*

# Storage Classes.

- Static Local Variables

- Declared with the keyword `static` are still `known only in the function` in which they're defined
- `Retain` their `value` when the function is exited
- The next time the function is called, the static local variable contains the value it had when the function last exited.
- The following statement declares local variable `count` to be static and initializes it to 1.  

```
static int count = 1;
```
- All numeric variables of static storage duration are initialized to zero by default if you do not explicitly initialize them.

# Scope Rules

- The scope of an identifier is the portion of the program in which the identifier can be referenced.
- For example, when we define a local variable in a block, it can be referenced only following its definition in that block or in blocks nested within that block.
- The four identifier scopes are function scope, file scope, block scope, and function-prototype scope.

# Scope Rules (cont'd)

- Labels with Function Scope

- Labels (identifiers followed by a colon such as `start:`) are the only identifiers with function scope.
- Labels can be used anywhere in the function in which they appear, but cannot be referenced outside the function body.
- Labels are used in `switch` statements (as case labels) and in `goto` statements (see Chapter 14).
- Labels are hidden in the function in which they're defined.
- This hiding—more formally called information hiding—is a means of implementing the principle of least privilege—a fundamental principle of good software engineering.
  - ◆ Code should be granted only the amount of privilege and access that it needs to accomplish its designated task, but no more.

# Scope Rules (cont'd)

- File Scope

- An identifier **declared outside any function** has file scope.
- Such an identifier is “known” (i.e., **accessible**) in all **functions** from the point at which the identifier is declared until the end of the file.
- Global variables, function definitions, and function prototypes placed outside a function all have file scope.

# Scope Rules (cont'd)

- Block Scope

- Identifiers defined inside a block have block scope.
- Block scope ends at the terminating right brace (}) of the block.
- Local variables defined at the beginning of a function have block scope as do function parameters, which are considered local variables by the function.
- Any block may contain variable definitions.
- When blocks are nested, and an identifier in an outer block has the same name as an identifier in an inner block, the identifier in the outer block is “hidden” until the inner block terminates.



# Scope Rules (cont'd)

- This means that while executing in the inner block, the inner block sees the value of its own local identifier and not the value of the identically named identifier in the enclosing block.
- Local variables declared static still have block scope, even though they exist from before program startup.
- Thus, storage duration does not affect the scope of an identifier.

# Scope Rules (cont'd)

- ```
for (int i = 0; i <= 10; i++) {  
    for (int j = 0; j <= 10; j++) {  
        int i = j;  
        if (i == 5) {  
            break;  
        }  
    }  
}
```



Common Programming Error 5.8

Accidentally using the same name for an identifier in an inner block as is used for an identifier in an outer block, when in fact you want the identifier in the outer block to be active for the duration of the inner block.



Error-Prevention Tip 5.4

Avoid variable names that hide names in outer scopes.

Scope Rules (cont'd)

- Function-prototype Scope

- The only **identifiers** with function-prototype scope are those used **in the parameter list of a function prototype**.
- As mentioned previously, function prototypes **do not require names** in the parameter list—**only types** are required.
- If a name is used in the parameter list of a function prototype, the **compiler ignores the name**.
- Identifiers used in a function prototype can be reused elsewhere in the program without ambiguity.

Scope Rules (cont'd)

- Scoping Issues

- We show the different scopes of global variables, automatic local variables, and static local variables in the following example program.

Scope Rules (cont'd)

```
1 // Scoping.
2 #include <stdio.h>
3
4 void useLocal(void); // function prototype
5 void useStaticLocal(void); // function prototype
6 void useGlobal(void); // function prototype
7
8 int x = 1; // global variable
9
10 int main(void)
11 {
12     int x = 5; // local variable to main
13
14     printf("local x in outer scope of main is %d\n", x);
15
16     { // start new scope
17         int x = 7; // local variable to new scope
18
19         printf("local x in inner scope of main is %d\n", x);
20     } // end new scope
21
22     printf("local x in outer scope of main is %d\n", x);
23
24     useLocal(); // useLocal has automatic local x
25     useStaticLocal(); // useStaticLocal has static local x
26     useGlobal(); // useGlobal uses global x
27     useLocal(); // useLocal reinitializes automatic local x
28     useStaticLocal(); // static local x retains its prior value
29     useGlobal(); // global x also retains its value
30
31     printf("\nlocal x in main is %d\n", x);
32
33     return 0;
34 }
```

local x in outer scope of main is 5
local x in inner scope of main is 7
local x in outer scope of main is 5

local x in useLocal is 25 after entering useLocal
local x in useLocal is 26 before exiting useLocal

local static x is 50 on entering useStaticLocal
local static x is 51 on exiting useStaticLocal

global x is 1 on entering useGlobal
global x is 10 on exiting useGlobal

local x in useLocal is 25 after entering useLocal
local x in useLocal is 26 before exiting useLocal

local static x is 51 on entering useStaticLocal
local static x is 52 on exiting useStaticLocal

global x is 10 on entering useGlobal
global x is 100 on exiting useGlobal

local x in main is 5

Scope Rules (cont'd)

```
36 // useLocal reinitializes local variable x during each call
37 void useLocal(void)
38 {
39     int x = 25; // initialized each time useLocal is called
40
41     printf("\nlocal x in useLocal is %d after entering useLocal\n", x);
42     ++x;
43     printf("local x in useLocal is %d before exiting useLocal\n", x);
44 }
45
46 // useStaticLocal initializes static local variable x only the first time
47 // the function is called; value of x is saved between calls to this
48 // function
49 void useStaticLocal(void)
50 {
51     // initialized once
52     static int x = 50;
53
54     printf("\nlocal static x is %d on entering useStaticLocal\n", x);
55     ++x;
56     printf("local static x is %d on exiting useStaticLocal\n", x);
57 }
58
59 // function useGlobal modifies global variable x during each call
60 void useGlobal(void)
61 {
62     printf("\nglobal x is %d on entering useGlobal\n", x);
63     x *= 10;
64     printf("global x is %d on exiting useGlobal\n", x);
65 }
```

Recursion

- The programs we've discussed are generally structured as functions that call one another in a disciplined, hierarchical manner.
- For some types of problems, it's useful to have functions call themselves.
- A recursive function is a function that calls itself either directly or indirectly through another function.
- Recursion is a complex topic discussed at length in upper-level computer science courses.
- In this section and the next, simple examples of recursion are presented.
- We consider recursion conceptually first, then examine several programs containing recursive functions.

Recursion (cont'd)

- Divide and Conquer via Recursion
 - Recursive problem-solving approach
 - A recursive function is called to solve a problem.
 - The function actually knows how to solve **only the simplest case(s)**, or so-called base case(s).
 - If the function is called with a **base case**, the function simply returns a result.
 - If the function is called with a more complex problem, the function **divides the problem into two conceptual pieces**: a piece that the function **knows how to do** and a piece that it does **not know how to do**.
 - To make recursion feasible, the latter piece **must resemble the original problem**, but be a slightly simpler or smaller version.

Recursion (cont'd)

- Because this new problem looks like the original problem, the function launches (calls) a fresh copy of itself to go to work on the smaller problem—this is referred to as a recursive call or the recursion step.
- The recursion step also includes the keyword return, because its result will be combined with the portion of the problem the function knew how to solve to form a result that will be passed back to the original caller.
- The recursion step executes while the original call to the function is paused, waiting for the result from the recursion step.

Recursion (cont'd)

- The recursion step can result in many more such recursive calls, as the function keeps dividing each problem it's called with into two conceptual pieces.
- For the recursion to terminate, each time the function calls itself with a slightly simpler version of the original problem, this sequence of smaller problems must eventually converge on the base case.
- When the function recognizes the base case, returns a result to the previous copy of the function, and a sequence of returns ensues all the way up the line until the original call of the function eventually returns the final result to main.

Recursion (cont'd)

- Recursively Calculating Factorials

- The factorial of a nonnegative integer n , written $n!$ (pronounced “ n factorial”), is the product

$$n \cdot (n - 1) \cdot (n - 2) \cdot \dots \cdot 1$$

with $1!$ equal to 1, and $0!$ defined to be 1.

- For example, $5!$ is the product $5 * 4 * 3 * 2 * 1$, which is equal to 120.

- The factorial of an integer, number, greater than or equal to 0 can be calculated **iteratively** (nonrecursively) using a for statement as follows:

```
factorial = 1;
for (counter = number; counter >= 1; counter--)
    factorial *= counter;
```

Recursion (cont'd)

- A recursive definition of the factorial function is arrived at by observing the following relationship:

$$n! = n \cdot (n - 1)!$$

or

$$a_n = n \cdot a_{n-1}$$

$$a_1 = 1 \text{ and } a_0 = 1$$

- For example, $5!$ is clearly equal to $5 \cdot 4!$ as is shown by the following:

$$5! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1$$

$$5! = 5 \cdot (4 \cdot 3 \cdot 2 \cdot 1)$$

$$5! = 5 \cdot (4!)$$

Recursion (cont'd)

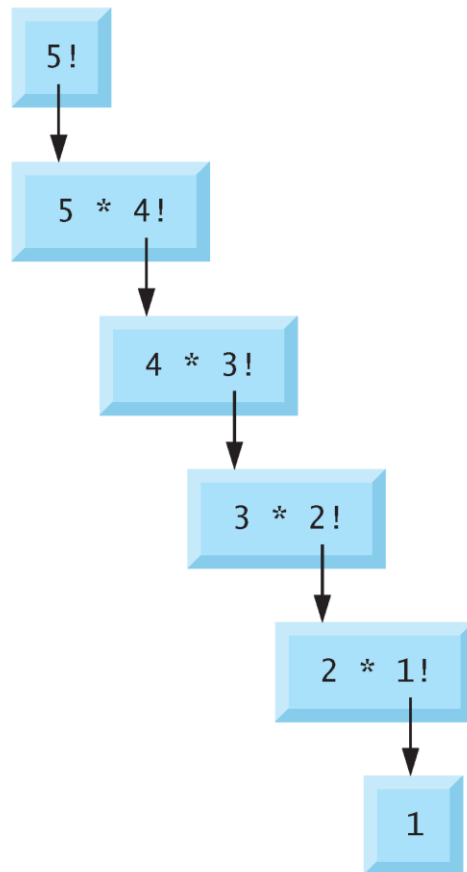
```
1 // Recursive factorial function.
2 #include <stdio.h>
3
4 unsigned long long int factorial(unsigned int number);
5
6 int main(void)
7 {
8     // during each iteration, calculate
9     // factorial(i) and display result
10    for (unsigned int i = 0; i <= 21; ++i) {
11        printf("%u! = %llu\n", i, factorial(i));
12    }
13
14    return 0;
15 }
16
17 // recursive definition of function factorial
18 unsigned long long int factorial(unsigned int number)
19 {
20     // base case
21     if (number <= 1) {
22         return 1;
23     }
24     else { // recursive step
25         return (number * factorial(number - 1));
26     }
27 }
```

```
0! = 1
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
6! = 720
7! = 5040
8! = 40320
9! = 362880
10! = 3628800
11! = 39916800
12! = 479001600
13! = 6227020800
14! = 87178291200
15! = 1307674368000
16! = 20922789888000
17! = 355687428096000
18! = 6402373705728000
19! = 121645100408832000
20! = 2432902008176640000
21! = 14197454024290336768
```

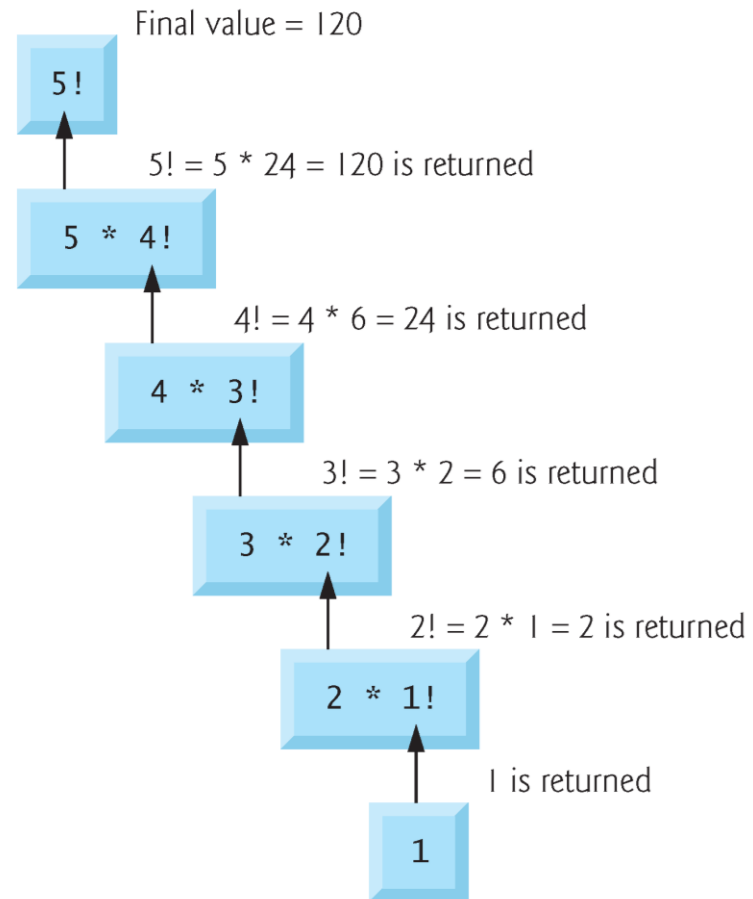
```
22! = 16127694944000000000
23! = 258520167388849920000
24! = 6204484017332398400000
25! = 15511210000000000000000
26! = 40329146112660563558400000
27! = 10888869440000000000000000
```

Recursion (cont'd)

a) Sequence of recursive calls



b) Values returned from each recursive call



Recursion (cont'd)

- If number is indeed less than or equal to 1, factorial returns 1, no further recursion is necessary, and the program terminates.
- If number is greater than 1, the statement

```
return number * factorial(number - 1);
```

expresses the problem as the product of number and a recursive call to factorial evaluating the factorial of `number - 1`.
- The call `factorial(number - 1)` is a slightly simpler problem than the original calculation `factorial(number)`.

Recursion (cont'd)

- Function factorial has been declared to receive a parameter of `type long` and return a result of type `long`.
 - ◆ Shorthand notation for `long int`
 - ◆ The C standard specifies that a variable of type `long int` is stored in `at least 4 bytes`, and thus may hold a value as large as +2147483647.
 - ◆ Factorial values become large quickly.
 - ◆ We've chosen the data type `long` so the program can calculate factorials greater than 7! on computers with small (such as 2-byte) integers.

Recursion (cont'd)

- The recursive factorial function first tests whether a terminating condition is true, i.e., whether number is less than or equal to 1.
- The conversion specifier `%llu` is used to print `unsigned long long int` values.
- Unfortunately, the factorial function produces large values so quickly that even unsigned long long int does not help us print very many factorial values before the maximum value of an unsigned long long int variable is exceeded.
- Even when we use `unsigned long long int`, we still **can't** calculate factorials **beyond 21!**

Recursion.

- This points to **a weakness in C** (and most other procedural programming languages)—namely that the language is **not easily extended to handle the unique requirements** of various applications.
- As we'll see later in the book, C++ is an extensible language that, through “classes,” allows us to create new data types, including ones that could hold arbitrarily large integers if we wish.



Common Programming Error 5.9

Forgetting to return a value from a recursive function when one is needed.



Common Programming Error 5.10

Either omitting the base case, or writing the recursion step incorrectly so that it does not converge on the base case, will cause infinite recursion, eventually exhausting memory. This is analogous to the problem of an infinite loop in an iterative (nonrecursive) solution.

Example Using Recursion: Fibonacci Series

- The Fibonacci series

0, 1, 1, 2, 3, 5, 8, 13, 21, ...

begins with 0 and 1 and has the property that each subsequent Fibonacci number is the sum of the previous two Fibonacci numbers.

- The series occurs in nature and, in particular, describes a form of spiral.
- The ratio of successive Fibonacci numbers converges to a constant value of 1.618....

$$\blacksquare \frac{1+\sqrt{5}}{2}$$

Example Using Recursion: Fibonacci Series (cont'd)

- This number, too, repeatedly occurs in nature and has been called the **golden ratio** or the **golden mean**.
- Humans tend to find the golden mean aesthetically pleasing.
- Architects often design windows, rooms, and buildings whose length and width are in the ratio of the golden mean.
- Postcards are often designed with a golden mean length/width ratio.

Example Using Recursion: Fibonacci Series (cont'd)

- The Fibonacci series may be defined recursively as follows:

$\text{fibonacci}(0) = 0$

$\text{fibonacci}(1) = 1$

$\text{fibonacci}(n) = \text{fibonacci}(n - 1) + \text{fibonacci}(n - 2)$

or

$F_n = F_{n-1} + F_{n-2}$
 $F_0 = 0$ and $F_1 = 1$

- Notice that Fibonacci numbers tend to become large quickly.
- Therefore, we've chosen the data type unsigned int for the parameter type and the data type unsigned long long int for the return type in function `fibonacci`.

Example Using Recursion: Fibonacci Series (cont'd)

```
1 // Recursive fibonacci function
2 #include <stdio.h>
3
4 unsigned long long int fibonacci(unsigned int n); // function declaration
5
6 int main(void)
7 {
8     unsigned int number; // number input by user
9
10    // obtain integer from user
11    printf("%s", "Enter an integer: ");
12    scanf("%u", &number);
13
14    // calculate fibonacci value for number input by user
15    unsigned long long int result = fibonacci(number);
16
17    // display result
18    printf("Fibonacci(%u) = %llu\n", number, result);
19
20    return 0;
21 }
22
23 // Recursive definition of function fibonacci
24 unsigned long long int fibonacci(unsigned int n)
25 {
26     // base case
27     if (0 == n || 1 == n) {
28         return n;
29     }
30     else { // recursive step
31         return fibonacci(n - 1) + fibonacci(n - 2);
32     }
33 }
```

Enter an integer: 0
Fibonacci(0) = 0

Enter an integer: 1
Fibonacci(1) = 1

Enter an integer: 2
Fibonacci(2) = 1

Enter an integer: 3
Fibonacci(3) = 2

Enter an integer: 10
Fibonacci(10) = 55

Enter an integer: 20
Fibonacci(20) = 6765

Enter an integer: 30
Fibonacci(30) = 832040

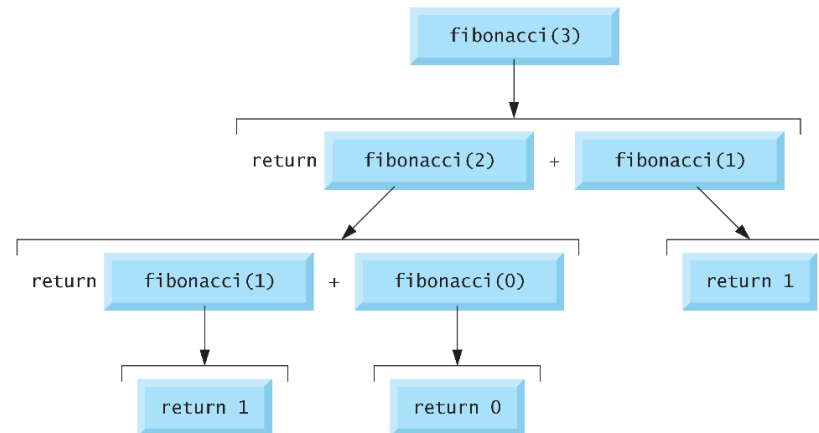
Enter an integer: 40
Fibonacci(40) = 102334155

Fibonacci(40) = 102334155
Enter an integer: 40

Fibonacci(30) = 832040
Enter an integer: 30

Example Using Recursion: Fibonacci Series (cont'd)

- The call to `fibonacci` from `main` is not a recursive call, but all subsequent calls to `fibonacci` are recursive
- Each time `fibonacci` is invoked, it immediately tests for the base case—`n` is equal to 0 or 1.
- If this is true, `n` is returned.
- Interestingly, if `n` is greater than 1, the recursion step generates two recursive calls, each a slightly simpler problem than the original call to `fibonacci`.



Example Using Recursion: Fibonacci Series (cont'd)

- Order of Evaluation of Operands

- This figure raises some interesting issues about the order in which C compilers will evaluate the operands of operators.
- This is a different issue from the order in which operators are applied to their operands, namely the order dictated by the rules of operator precedence.
- While evaluating `fibonacci(3)`, two recursive calls will be made, namely `fibonacci(2)` and `fibonacci(1)`.
- But **in what order** will these calls be made? You might simply assume the operands will be evaluated left to right.

Example Using Recursion: Fibonacci Series (cont'd)

- For optimization reasons, C does not specify the order in which the operands of most operators (including +) are to be evaluated.
- Therefore, you should make no assumption about the order in which these calls will execute.
- The calls could in fact execute `fibonacci(2)` first and then `fibonacci(1)`, or the calls could execute in the reverse order, `fibonacci(1)` then `fibonacci(2)`.
- In this program and in most other programs, the final result would be the same.

Example Using Recursion: Fibonacci Series (cont'd)

- But in some programs the evaluation of an operand may have side effects that could affect the final result of the expression.
- C specifies the order of evaluation of the operands of only four operators—namely `&&`, `||`, the comma `(,)` operator and `? : .`
- The first three of these are binary operators whose operands are guaranteed to be evaluated left to right.
 - ◆ [Note: The commas used to separate the arguments in a function call are not comma operators.]
- The last operator is C's only ternary operator.

Example Using Recursion: Fibonacci Series (cont'd)

■ In ternary operator

- ◆ Its leftmost operand is always evaluated first.
- ◆ If the leftmost operand evaluates to nonzero, the middle operand is evaluated next and the last operand is ignored.
- ◆ If the leftmost operand evaluates to zero, the third operand is evaluated next and the middle operand is ignored.



Common Programming Error 5.11

Writing programs that depend on the order of evaluation of the operands of operators other than `&&`, `||`, `?:`, and the comma `(,)` operator can lead to errors because compilers may not necessarily evaluate the operands in the order you expect.



Portability Tip 5.2

Programs that depend on the order of evaluation of the operands of operators other than `&&`, `||`, `?:`, and the comma `(,)` operator can function differently on different compilers.

Example Using Recursion: Fibonacci Series (cont'd)

- Exponential Complexity

- A word of caution is in order about recursive programs like the one we use here to generate Fibonacci numbers.
- Each level of recursion in the `fibonacci` function has a doubling effect on the number of calls—the number of recursive calls that will be executed to calculate the n^{th} Fibonacci number is on the order of 2^n .
- This rapidly gets out of hand.
- Calculating only the 20th Fibonacci number would require on the order of 2^{20} or about a million calls, calculating the 30th Fibonacci number would require on the order of 2^{30} or about a billion calls, and so on.

Example Using Recursion: Fibonacci Series.

- Computer scientists refer to this as **exponential complexity**.
- Problems of this nature humble even the world's most powerful computers!
- Complexity issues in general, and exponential complexity in particular, are discussed in detail in the upper-level computer science curriculum course generally called “Algorithms.”
- The example we showed in this section used an intuitively appealing solution to calculate Fibonacci numbers, but there are better approaches.

Example Using Recursion: Recursive main

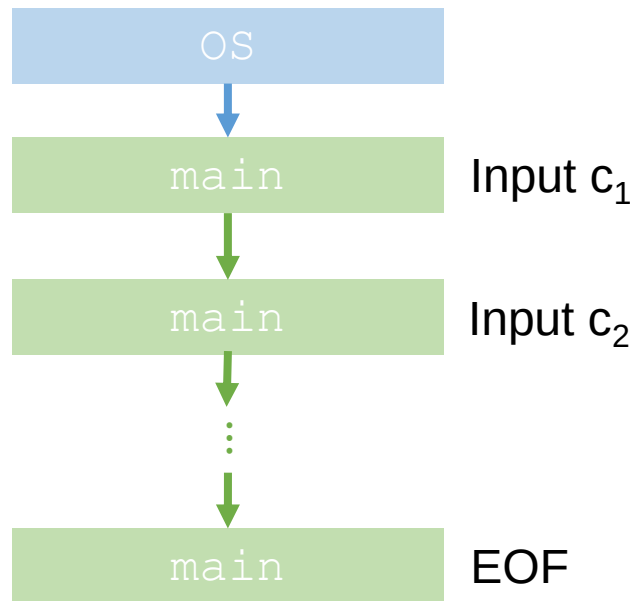
- Considering the following code:

```
1  #include <stdio.h>
2
3  int main(void)
4  {
5      int c; // variable to hold character input by user
6
7      if ((c = getchar()) != EOF) {
8          main();
9          printf("%c", c);
10     }
11 }
```

- What does the program do?

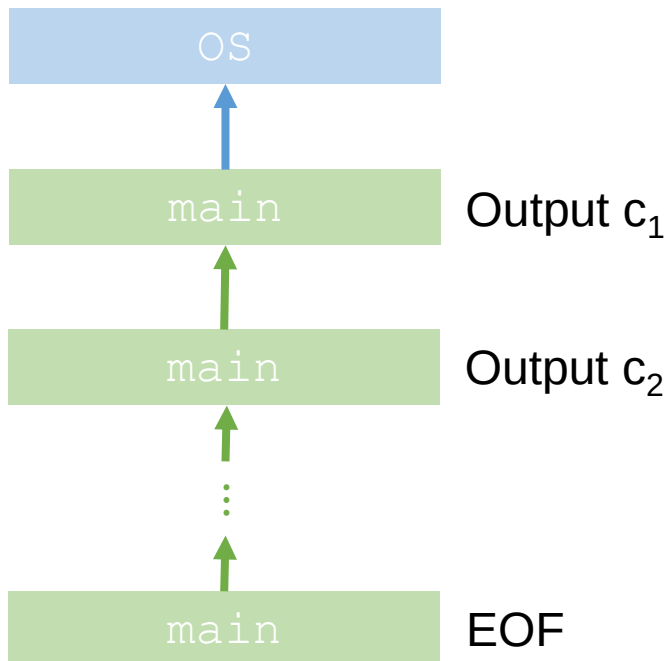
Example Using Recursion: Recursive main (cont'd)

- The OS calls the function main at first.
- In the main function, if the user input character is not EOF, then the other main function is called. The second called main function will wait the user input like the first main function.



Example Using Recursion: Recursive main (cont'd)

- The last called main function which encounters EOF returns first, and then the second last one, and so forth.
- The input character will be output before the main function return.



Example Using Recursion: Recursive main.

- Therefore, the input

a
b
c
d
e
^z

has the output

e
d
c
b
a

Example Using Recursion: Simple Recursion Formula

- Considering the following recursive function:

```
// Parameter b must be a positive integer
// to prevent infinite recursion
unsigned int mystery(unsigned int a, unsigned int b)
{
    // base case
    if (1 == b) {
        return a;
    }
    else { // recursive step
        return a + mystery(a, b - 1);
    }
}
```

- What does the program do?

Example Using Recursion: Simple Recursion Formula (cont'd)

- The recursive function can be separated into two cases:

- ◆ $b == 1$, the base case
- ◆ Otherwise.

- Let the result of

`mystery(a, b)`

be $m_{a,b}$

- For the base case

$$m_{a,1} = a$$

`return a;`

- For all the other cases

$$m_{a,b} = a + m_{a,b-1}$$

`return a + mystery(a, b - 1);`

Example Using Recursion: Simple Recursion Formula (cont'd)

- Solve the recursive formula

$$\begin{aligned}m_{a,b} &= a + m_{a,b-1} \\&= a + (a + m_{a,b-2}) \\&= a + a + (a + m_{a,b-3}) \\&= a + \dots + a + m_{a,1} \\&= a + \dots + a + a\end{aligned}$$

- How many 'a's?

- ◆ Observe the indices b
- ◆ $b - 1$: 1 'a', $b - 2$: 2 'a's, $b - k$: k 'a's
- ◆ $b - k = 1$:
 $k = b - 1$, so with $m_{a,1}$ ($=a$) we have b 'a's
- ◆ $m_{a,b} = b \times a = ba$

Example Using Recursion: Simple Recursion Formula (cont'd)

- The $\text{pow}(\text{base}, \text{exponent})$ returns the value of $\text{base}^{\text{exponent}}$
- Such calculation can be made by the recursive function.
- The formula $\text{base}^{\text{exponent}}$ can be re-written as $\text{base} \times \text{base} \times \dots \times \text{base}$ which equals to $\text{base}^{\text{exponent}-1} \times \text{base}$.
- This means $\text{recursivePow}(\text{base}, \text{exponent}) = \text{recursivePow}(\text{base}, \text{exponent} - 1) \times \text{base}$
- The boundary condition would be $\text{recursivePow}(\text{base}, 0) = 1$

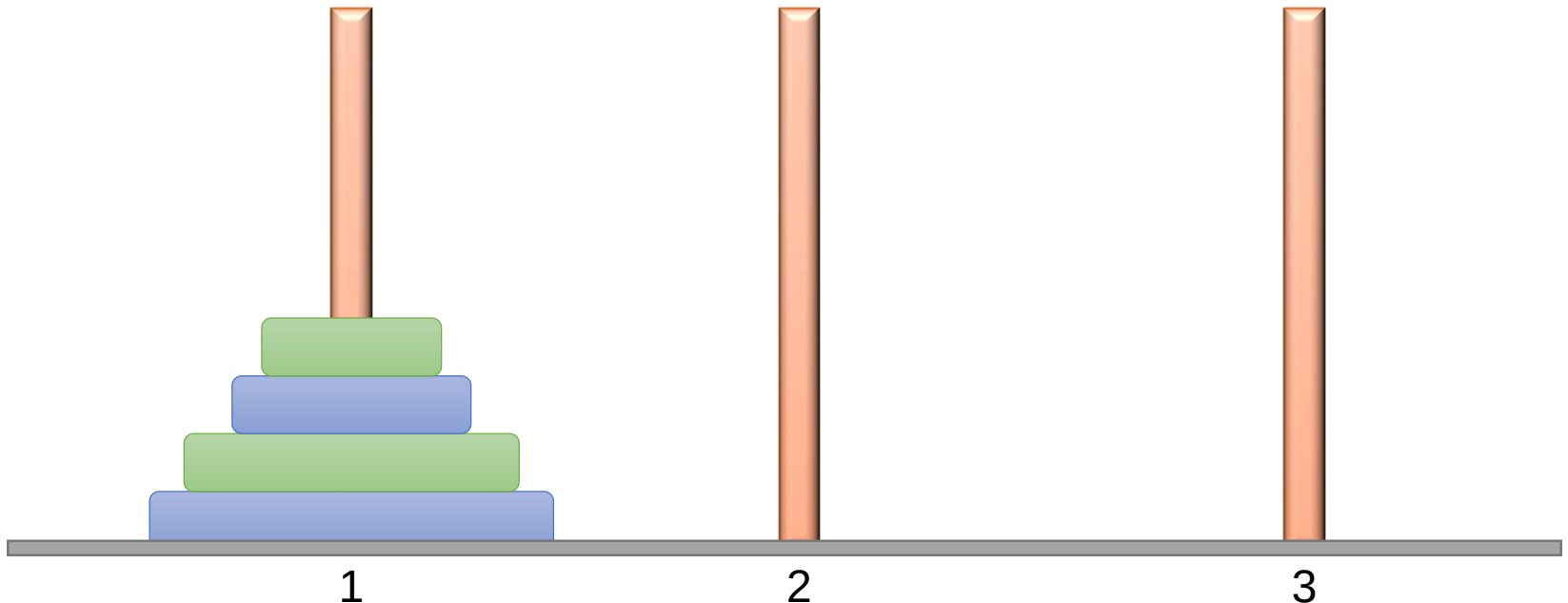
Example Using Recursion: Simple Recursion Formula.

- The recursive function is

```
double recursivePow(double base, int exponent)
{
    if (exponent == 0)
    {
        return 1;
    }
    else
    {
        return recursivePow(base, exponent - 1) * base;
    }
}
```

Example Using Recursion: Towers of Hanoi

- The priest wants to move the stack from one peg to another peg
- Each time we can only move one disk
- At any time, a larger disk cannot put on a smaller one.



Example Using Recursion: Towers of Hanoi (cont'd)

- The problem can be separated into three processes
 - Move the upper $n - 1$ disks from peg1 to peg2
 - Move the largest disk from peg1 to peg3
 - Move the $n - 1$ disks from peg2 to peg3
- So the recursion will be
 - Let $S_{n,s,d,t}$ be a solution of Hanoi tower with n disks from source peg s to destination peg d through peg t , and $M_{n,s,d}$ denote the movement of n_{th} disk from source peg s to destination peg d .
 - We have: $S_{n,s,d,t} = S_{n-1,s,t,d} + M_{n,s,d} + S_{n-1,t,d,s}$
- The base case (boundary condition) is
 - $S_{1,s,t,d} = M_{1,s,d}$

Example Using Recursion: Towers of Hanoi.

- From the recursive step and base case

$$S_{n,s,d,t} = S_{n-1,s,t,d} + M_{n,s,d} + S_{n-1,t,d,s}$$

$$S_{1,s,t,d} = M_{1,s,d}$$

the recursive function `solveHanoi` can be written as

```
void solveHanoi(int n, int s, int d, int t)
{
    solveHanoi(n - 1, s, t, d);
    move(n, s, d);
    solveHanoi(n - 1, t, d, s);
}
```

and the function `move` prints the corresponding movements

```
void move(int n, int s, int d)
{
    printf("Move disk %d from %d to %d", n, s, d);
}
```

- Call `solveHanoi(n, 1, 3, 2)` to get the solution of towers of Hanoi.

Example Using Recursion: Greatest Common Divisor

- The greatest common divisor (GCD) of integer x and y is the largest integer that evenly divides both x and y .
- The characteristics of GCD is that
 - ◆ GCD of x and y is x if y equals to zero (base case)
 - ◆ Otherwise, GCD of x and y is GCD of y and $x \bmod y$
- So the recursive function `gcd` for GCD is

```
int gcd (int x, int y)
{
    if ( y == 0 )
        return x;
    else
        return gcd (y, x % y);
}
```

Recursion vs. Iteration

- Both iteration and recursion are based on a control statement
 - Iteration uses a repetition statement.
 - Recursion uses a selection statement.
- Both iteration and recursion involve repetition: Iteration explicitly uses a repetition statement; recursion achieves repetition through repeated function calls.
- Iteration and recursion each involve a termination test
 - Iteration terminates when the loop-continuation condition fails.
 - Recursion terminates when a base case is recognized.

Recursion vs. Iteration (cont'd)

- Iteration with counter-controlled repetition and recursion each gradually approach termination: Iteration keeps modifying a counter until the counter assumes a value that makes the loop-continuation condition fail; recursion keeps producing simpler versions of the original problem until the base case is reached.
- Both iteration and recursion can occur infinitely:
 - An **infinite loop** occurs with iteration if the loop-continuation test never becomes false
 - **Infinite recursion** occurs if the recursion step does not reduce the problem each time in a manner that converges on the base case.
 - Infinite iteration and recursion typically occur as a result of **errors in a program's logic**.

Recursion vs. Iteration (cont'd)

- Recursion has many negatives.
 - It repeatedly invokes the mechanism, and consequently the **overhead, of function calls**.
 - This can be expensive in both **processor time and memory space**.
 - Each recursive call causes another copy of the function (actually only the function's variables) to be created; this can consume considerable memory.
 - Iteration normally occurs within a function, so the overhead of repeated function calls and extra memory assignment is omitted.
 - So why choose recursion?



Software Engineering Observation 5.12

Any problem that can be solved recursively can also be solved iteratively (nonrecursively). A recursive approach is normally chosen in preference to an iterative approach when the recursive approach more naturally mirrors the problem and results in a program that's easier to understand and debug. Another reason to choose a recursive solution is that an iterative solution may not be apparent.

Recursion vs. Iteration (cont'd)

- Good software engineering is important
 - ◆ Good software engineering is key to making more manageable the task of developing the larger and more complex software systems we need.
- High performance is also important
 - ◆ High performance is key to realizing the systems of the future that will place ever greater computing demands on hardware.
- Unfortunately, these goals are often at odds with one another.

Recursion vs. Iteration.

■ Where do functions fit in here?



Performance Tip 5.2

Dividing a large program into functions promotes good software engineering. But it has a price. A heavily functionalized program—as compared to a monolithic (i.e., one-piece) program without functions—makes potentially large numbers of function calls, and these consume execution time on a computer’s processor(s). Although monolithic programs may perform better, they’re more difficult to program, test, debug, maintain, and evolve.



Performance Tip 5.3

Today’s hardware architectures are tuned to make function calls efficient, C compilers help optimize your code and today’s hardware processors are incredibly fast. For the vast majority of applications and software systems you’ll build, concentrating on good software engineering will be more important than programming for high performance. Nevertheless, in many C applications and systems, such as game programming, real-time systems, operating systems and embedded systems, performance is crucial, so we include performance tips throughout the book.

Improve Recursion using Tail recursion

- The overhead of function call is severe in certain cases.
- Compiler are improved to handle and optimize function call.
- A special recursion called tail recursion can be optimized by compiler and bring about efficiency.
- The function call right before the end of a function is known as tail call.
- If the called function is itself, it's a tail recursion.

Improve Recursion using Tail recursion.

- The recursive greatest common divisor is a tail recursion:

```
int gcd (int x, int y)
{
    if ( y == 0 )
        return x;
    else
        return gcd (y, x % y);
}
```

Improve Recursion using Tail recursion: Fibonacci

- The Fibonacci function can also be written as tail recursion:

```
unsigned long long int tailFibonacci(int n, unsigned long long fib1,
                                     unsigned long long int fib 2)
{
    if ( n > 0 )
        return tailFibonacci(n - 1, fib2, fib1 + fib2);
    else
        return fib1;
}
```

- Function call `tailFibonacci(n, 0, 1)` returns the n_{th} Fibonacci number.

Improve Recursion using Tail recursion: Fibonacci.

- `tailFibonacci (5, 0, 1)`
- `tailFibonacci (4, 1, 1)`
- `tailFibonacci (3, 1, 2)`
- `tailFibonacci (2, 2, 3)`
- `tailFibonacci (1, 3, 5)`
- `tailFibonacci (0, 5, 8)`
- `return 5`

Exercises

- What is argument coercion? When will it happen?
- What are the three ways of returning control back to the calling function?
- Briefly describe the two methods passed by value and passed by reference when passing arguments.

Exercises

- What are the two main functions of function call stack and stack frame when calling functions?
- Explain the behavior of the following statement:
`int c = a + rand() % b;`

Exercises

- What will happen when a called function completes its task?
- Write a function which returns the minimum of four characters.

Exercises

- How to round a floating-point value x to the hundred and thousand position, and store it to an integer y ?
- Write a function to calculate and returns the final position of an object with three parameters speed v , acceleration a , and time t using the following formula: $s = vt + \frac{1}{2}at^2$. Remember to use appropriate data type.

Exercises

- Unexpectedness is important in many applications. Please give a method in C to randomize your random number generator which may change the random sequence over time.
- How to generate a random value between 4 and 27 (inclusive). How to generalize it if we replace 4, and 27 by two integers a and b?

Exercises

- Storage class influences the storage duration and scope of a variable. Give a definition for each of storage duration and scope.

- Explain the behavior of the following code:

```
#include <stdio.h>
int main(void)
{
    int c;
    if ( ( c = getchar() ) != EOF ) {
        main();
        printf("%c", c);
    }
    return 0;
}
```

Exercises

- Consider the following code:

```
1  #include <stdio.h>
2  int x = 10; // (a) for x
3  int add(int x, int y); // (b) for add, (c) for x
4  int main(void)
5  {
6      double x = 2.34, y = 4.56 // (d) for x
7      if ( (x = 2) || (y = 5) ) {
8          int z = add(x, y); // (e) for z
9      }
10     int w = add(x, y);
11     //here
12     return 0;
13 }
14 int add(int num1, int num2) {
15     static int base; // (f) for base
16     base++;
17     x = num1 + num2 + base;
18     return x;
19 }
```

- What is the final value of x, y, z and w at line 11?
- Give the storage class (auto, static, extern, register), storage duration (automatic, static) and scope (function, function prototype, file, block) of identifiers in lines 2, 3, 6, 8, and, 15.

Exercises

- The greatest common divisor (GCD) of two numbers x and y is x if $y = 0$; otherwise it equals to the GCD of y , and $x \bmod y$. Write the recursive function for finding the GCD of two numbers.

Exercises

- What does the following recursive function do?

```
unsigned int mystery(unsigned int a,  
unsigned int b) {  
    if ( a >= b ) {  
        return a;  
    }  
    else {  
        return a + mystery(a+1, b-1);  
    }  
}
```

You have to

- a) apply the recursive formula for `mystery(a, b)` until the base case, and
- b) show the results of “`mystery(4, 15);`”